data ...../...../.....
S T Q Q S S D

Fase 1 questão 1:

```
const lista = [3, 7, 2, 9];
lista.push(5);
console.log(lista.length);
// o resultado foi 5, pois o length não lê por indices e
sim pela quantidade de elementos.
```

Fase 1 questão 2

```
const frutas = 'maçã', 'banana', 'uva'];
frutas.pop();
console.log(frutas);
// pop() remove no final da lista R: ["maçã", banana]
```

Fase 1 questão 3.

```
let numeros = [10, 20, 30, 40];
numeros.shift();
console.log(numeros);
// remove o primeiro elemento da lista. R: [20, 30, 40];
```

Fase 1 questão 4

```
const valores = [1, 2, 3];
valores.unshift(0);
console.log(valores);
// Adiciona no primeiro elemento da lista. R: [0, 1, 2, 3]
```

Fase 1 questão 5

```
const a = [5, 6, 7];        R: [6, 7]
const b = a.slice(1);
console.log(b);
// Ele pega os elementos a partir do indice determinado
```

Fase 1 questão 6

```
const arr = [1, 2, 3, 4];   R: [3, 4]
arr.splice(1, 2);
console.log(arr);
// remove dois elementos do array a partir do indice 1.
```

Fase 1 Questão 7

```
const lista = ['a','b','c'];        R: ['c']
Console.log(lista.indexOf('c'));
//retorna o indice onde o elemento 'c' está localizado no array.
```

Fase 1 Questão 8

```
const itens [4,8,12];
const x = itens.map(x => x/2);      R [8,16,24];
Console.log(x);
// Cria um novo array com todos os valores divididos por 2.
```

Fase 1 Questão 9

```
const numeros = [3,6,9,12];
const filtrado = numeros.filter(n => n > 6);
Console.log(filtrado)               R: ['9',12];
```

Fase 1 Questão 10

```
const arr = [2,4,6];
const soma = arr.reduce((a,b) => a+b, 0)
Console.log(soma);  R [12]
// Soma todos os valores do array e retorna o resultado
```

Fase 1 Questão 15

```
const nomes = ['Ana','Lucas','Bruna'];
Console.log(nomes.includes('Lucas'));  R: True
//Verifica se o valor 'Lucas' existe no array.
```

Fase 1 Questão 12.

```
const letras = ['a','b','c'];  R: a-b-c
Console.log(letras.join('-'));
// Junta ou concatena os elementos do array em uma única string separado por '-'
```

Fase 1 Questão 13

```
let arr = [1, 2, 3];        R [1, 2, 3, 4, 5]
arr = arr.concat([4, 5]);
console.log(arr);
// Junta dois arrays e retorna um novo array com todos os
elementos.
```

Fase 1 Questão 14

```
const lista = ['x', 'y', 'z'];   //R: [~~'x', 'y', 'z'~~]; ['z', 'y', 'x']
console.log(lista.reverse())
// Inverte a ordem dos elementos do array
```

Fase 1 Questão 15

```
const dados = [1, 2, 3, 4];   R = 3
const n = dado.find(m => m > 2);
console.log(n);
// Retorna o primeiro elemento do array que for maior que 2.
```

Fase 2 Questão 1

```
const arr = [1, 2, 3, 4, 5];
const resultado = arr.filter(m => m % 2 !== 1).map(m => m * 3).
reduce(acc, v) => acc + v, 10);   R = 37
console.log(resultado);
// Filtra os numeros impares, multiplica cada um por 3 e
soma tudo começando em 10.
```

Fase 2 Questão 2

```
const arr = [10, 15, 22, 34, 45, 60];
const processado = arr.filter(m => m % 3 === 0).map(m => ({ orig-
inal: m, metade: m / 2 })).reduce((acc, obj) => acc + obj.me-
tade, 0);
console.log(processado)
```

Resposta.

```
const arr = [30, 35, 22, 34, 45, 60];
let soma = 0;
for (let i = 0; i < arr.length; i++){
    const n = arr[i];
  if (n % 3 === 0){   // Equivalente ao filter
      const obj = { // Equivalente ao map
        original: n,
        metade: n/2
      };
    soma += obj.metade; // Equivalente ao reduce
  }
}
console.log(soma); R: 60
```

Forma 2 Questão 3

```
const lista = [4, 8, 12, 16];
const r = lista.slice(1, 2);
console.log(lista, r);
```

outros metodos:

```
const lista = [4, 8, 12, 16];
const r = lista.slice(8, 16);
console.log(lista, r);
const copiaLista = [...lista];
console.log(copiaLista);
// slice e spread conseguem criar a copia que foi pedida
```

Forma 2 Questão 4

```
const pessoas = [{ nome: 'Ana', cidade: 'SP' }, { nome: 'Lucas',
cidade: 'RJ' }, { nome: 'Bruno', cidade: 'SP' }, { nome: 'Caio', cidade:
'MG' }];
```

```
const resultado = pessoas.reduce((acc, pessoa) => {
    const cidade = pessoa.cidade
  if (!acc[cidade]){
      acc[cidade] = [];
  }
  acc[cidade].push(pessoa);
  return acc;
  }, {});
console.log(resultado);
```

R {
SP: [
{nome: 'Ana', cidade: 'SP'},
{nome: 'Bruno', cidade: 'SP'}
],
RJ: [{nome: 'Lucas', cidade: 'RJ'}
],
MG: [
{nome: 'Caio', cidade: 'MG'}
]
}

//usando reduce do para agrupar as pessoas por cidade em um objeto

Fase 2 Questão 5

```
const valores = [5, 8, 12, 8, 3];
const a = valores.find(v => v === 8);
const b = valores.filter(v => v === 8);
const c = valores.some(v => v === 8);
console.log(a, b, c);
```

R: 8 [8, 8] True

//Busca o primeiro 8, cria um array com todos os 8 e verifica se existe algum 8 no array;

Fase 3 Questão 1

```
const usuario = { nome: "Ana", idade: 28, cidade: "SP" };
const { nome, ...resto } = usuario;
```

//a) "Ana" { idade 28, cidade "SP" }

// Porque o destructing separa o nome e o operador ...resto guarda as demais propriedades do objeto.

Fase 3 Questão 2

```
const numeros = [3, 2, 7, 4, 5];
const [a, , b, c] = numeros;
console.log(a, b, c);
```

//a) 3 [4, 5]

// Porque o destructing pega 3 em 'a', pula 2, pega 7 em 'b' e o restante vai para 'c'.

data ..... / .....
S T Q Q S

Fase 3 questão 3

```
const pessoa = { nome: "Luiz", endereço: { cidade: "RJ", cep: 3334
}};
const { endereço: { cidade } } = pessoa;
console.log(cidade);
```

// a) "RJ"

// Porque o destructing pessoa diretamente o pra propriedade 'cidade' dentro de endereço

Fase 3 questão 4

```
const base = { a: 1, b: 2 };
const extra = { b: 3, c: 4 };
const combinado = { ...base, ...extra };
console.log(combinado);
```

// b) { a: 1, b: 3, c: 4 }

// Porque o spread copia os objetos e o valor de b em extra sobrescreve o de base.

Fase 3 questão 5

```
function Teste({x = 10, y = 20} = {}) {
  console.log(x+y);
}
Teste({x = 5});
```

// b 25

// Porque x recebe 5 e y usa o valor padrão 20.

Fase 3 questão 6

~~Consol~~

```
const arr1 = [1, 2]; const arr2 = [3, 4];
const arr3 = [...arr1, ...arr2, 5];
```

// c) [1, 2, 3, 4, 5]

// porque o spread espalha os elementos dos arrays dentro de um novo array.

data ...../...../.....
S T Q Q S S D

Fase 3 Questão 7

```
const config = { tema: "Escuro", fonte "Arial", tamanho: 14 };
const { tema: modo ... opcoes } = config;
console.log (modo, opcoes.fonte);
// a) "Escuro" "Arial"
// Porque 'tema' é renomeado para 'modo' e o restante fica em 'opcoes'
```

Fase 3 Questão 8

```
const valores = [10, 20, 30, 40];
function soma (a, b ... resto) {
    return a + b + resto.length;
}
console.log (soma(...valores));
// b) 32
// Porque a = 10, b = 20 e 'resto' recebe [30, 40], que possui 2 elementos.
```

Fase 3 Questão 9

```
const usuario = {
  nome: 'Carlos,
  habilidades: ["JS", "Python", "SQL"]
};
const { habilidades: [h1, , h3] } = usuario;
console.log (h1, h3);
// a) 'JS' 'SQL'
// Porque o destructing pega o primeiro e terceiro elementos do array 'habilidades'
```

Fase 3 Questão 10

```
const obj1 = { a: 1, b: { x: 10 } };
const obj2 = { ... obj1 };
console.log (obj1.b.x);
// b) 99
// Porque o spread fez uma cópia rasa e o objeto interno 'b' continua compartilhado.
```

Fazer 4 exercício 1

```
const matriz = [[1,2],[3,4],[5,6]];
const [x, [y, z], ...resto] = matriz;
console.log(y, resto);
//b) 4 [[5,6]]
//Porque o destructing ignora o primeiro array, pega o 4 do segundo e o restante vai para 'resto'
```

Fazer 4 exercício 2

```
const obj = {
id 1,
dados{
nome: "Ana",
preferencias:{
tema: "escuro",
fonte: "sans"
}
  }
}
const {dados: {preferencias: {tema}, ...resto}} = obj;
console.log(tema, resto);
//a) "escuro" {nome: "Ana"}
//Porque 'tema' é extraído de preferencias e o restante dos dados fica em resto.
```

Fazer 4 exercício 3

```
const dados = {a: 1, b: 2, c: 3};
function processar({a, ...resto}){
  resto.c = 99
  return [a, resto];
} //a) 1 99
//Porque 'resto' é uma cópia das propriedades restantes e alterar 'resto.c' não muda o objeto original.
```

Fase 4 Questão 4

```
const arr = [1, 2, 3];
const func = ([x, ...resto], y = [...resto, x]) => {
    return y;
};
console.log(func(arr));
// a) [2, 3, 1]
// Porque x recebe 1, resto recebe [2, 3] e y usa o valor
padrão [...resto, x]
```

Fase 4 Questão 5

```
const config1 = {mode: "dark", cores: {layout: "grid", zoom: 100}};
const config2 = {...config1, cores: {...config1.cores}};
config2.cores.zoom = 200;
console.log(config1.cores.zoom, config2.cores.zoom);
// e) 100 200
// Porque "cores" também foi copiado com spread, criando
um novo objeto independente.
```